

Syntax Directed Translation

KC Sivaramakrishnan

IIT Madras

Syntax-Directed Translation

A parser discovers a program's structure.

How can that structure guide a translation?

Infix input

3 + 5

Postfix output

3 5 +

A translation rule for addition:

$$E \rightarrow E_1 + T \quad E.code = E_1.code \parallel T.code \parallel ' + '$$

$\parallel$ concatenates the postfix strings.

Each expression node has a *code* **attribute**.

The production's **semantic rule** combines its children's code.

What determines the order of these computations?

Syntax-directed definition (SDD)

- A context-free grammar with *attributes* and *semantic rules*.
- Attributes belong to grammar symbols. Rules attached to productions specify how to compute their values.

Syntax-directed translation scheme (SDT)

- A context-free grammar with executable *semantic actions* embedded in production bodies.
- An action's position specifies when it executes relative to the symbols and other actions in that production.

An **SDD** specifies the computation.

An **SDT** makes the execution order explicit.

Infix to Postfix: SDD and SDT

3 - 5 + 2 has postfix form 3 5 - 2 +

SDT scheme

$$E \rightarrow E_1 + T \{\text{print } '+'\}$$
$$E \rightarrow E_1 - T \{\text{print } '-'\}$$
$$E \rightarrow T$$
$$T \rightarrow 0 \{\text{print } '0'\}$$
$$T \rightarrow 1 \{\text{print } '1'\}$$
$$\dots$$
$$T \rightarrow 9 \{\text{print } '9'\}$$

SDD

$$E \rightarrow E_1 + T \quad E.code = E_1.code \| T.code \| '+'$$
$$E \rightarrow E_1 - T \quad E.code = E_1.code \| T.code \| '-'$$
$$E \rightarrow T \quad E.code = T.code$$
$$T \rightarrow 0 \quad T.code = '0'$$
$$T \rightarrow 1 \quad T.code = '1'$$
$$\dots$$
$$T \rightarrow 9 \quad T.code = '9'$$

Infix to Prefix: SDD and SDT

3 - 5 + 2 has prefix form + - 3 5 2

SDT scheme

$$E \rightarrow \{\text{print '+'}\} E_1 + T$$
$$E \rightarrow \{\text{print '-'}\} E_1 - T$$
$$E \rightarrow T$$
$$T \rightarrow 0 \{\text{print '0'}\}$$
$$T \rightarrow 1 \{\text{print '1'}\}$$
$$\dots$$
$$T \rightarrow 9 \{\text{print '9'}\}$$

SDD

$$E \rightarrow E_1 + T \quad E.\text{code} = ' + ' \| E_1.\text{code} \| T.\text{code}$$
$$E \rightarrow E_1 - T \quad E.\text{code} = ' - ' \| E_1.\text{code} \| T.\text{code}$$
$$E \rightarrow T \quad E.\text{code} = T.\text{code}$$
$$T \rightarrow 0 \quad T.\text{code} = ' 0 '$$
$$T \rightarrow 1 \quad T.\text{code} = ' 1 '$$
$$\dots$$
$$T \rightarrow 9 \quad T.\text{code} = ' 9 '$$

Syntax directed translation - Parse tree

Idiomatic syntax directed translation does the following:

- 1 Construct a parse tree
- 2 Compute the values of the attributes at the nodes of the tree by visiting the tree

Key: We don't need to build a parse tree all the time.

- Translation can be done during parsing.

Attributes

- Attribute is any quantity associated with a programming construct.
- Example: data types, line numbers, instruction details

Two kinds of attributes: for a non-terminal A , at a parse tree node N

- *A synthesized attribute*:
 - defined by a semantic rule associated with the production at N .
 - defined only in terms of attribute values at the children of N and at N itself.
- *An inherited attribute*:
 - defined by a semantic rule associated with the parent production of N .
 - defined only in terms of attribute values at the parent of N , siblings of N and at N itself.

Specifying the actions: Attribute grammars

Donald Knuth, 1968

Semantics of Context-Free Languages

Idea: attribute the parse tree

- can add attributes (*fields*) to each node
- specify equations to define values (unique)
- can use attributes from parent and children

Example: to ensure that constant variables are immutable:

- add *type* (`int`, `bool`, ...) and *kind* (`var`, `const`) attributes expression nodes.
- rules for production on `:=` (assignment) that
 - 1 check that LHS *kind* is `var`
 - 2 check that LHS *type* and RHS *type* are consistent or conform

Attribute grammars

Formally, we define the notion of *attribute grammars*:

- grammar-based specification of parse-tree attributes
- value assignments associated with productions
- each attribute uniquely, locally defined
- label identical terms uniquely

Can specify context-sensitive actions with attribute grammars

Type Declarations: Passing a Type to a List

The grammar generates `int` or `real`, followed by one or more comma-separated identifiers.

Examples: `int x, y, z` `real a`

Goal: record the declared type in each identifier's symbol-table entry.

PRODUCTION	SEMANTIC RULES
$D \rightarrow T L$	$L.in := T.type$
$T \rightarrow \mathbf{int}$	$T.type := \text{integer}$
$T \rightarrow \mathbf{real}$	$T.type := \text{real}$
$L \rightarrow L_1, \mathbf{id}$	$L_1.in := L.in$ $\text{addtype}(\mathbf{id}.entry, L.in)$
$L \rightarrow \mathbf{id}$	$\text{addtype}(\mathbf{id}.entry, L.in)$

id is an identifier token. $L.in$ carries the declared type.

Is $L.in$ Inherited or Synthesized?

Consider the rules from the declaration grammar:

$$D \rightarrow T L \qquad L.in := T.type$$

$$L \rightarrow L_1, \mathbf{id} \qquad L_1.in := L.in$$

Which kind of attribute is $L.in$? Why?

Identify the node whose attribute each rule defines, and where its value comes from.

Inherited.

- In $D \rightarrow T L$, L receives the type from its sibling T .
- In $L \rightarrow L_1, \mathbf{id}$, L_1 receives it from its parent L .

Both rules define an attribute of a child in its parent's production.

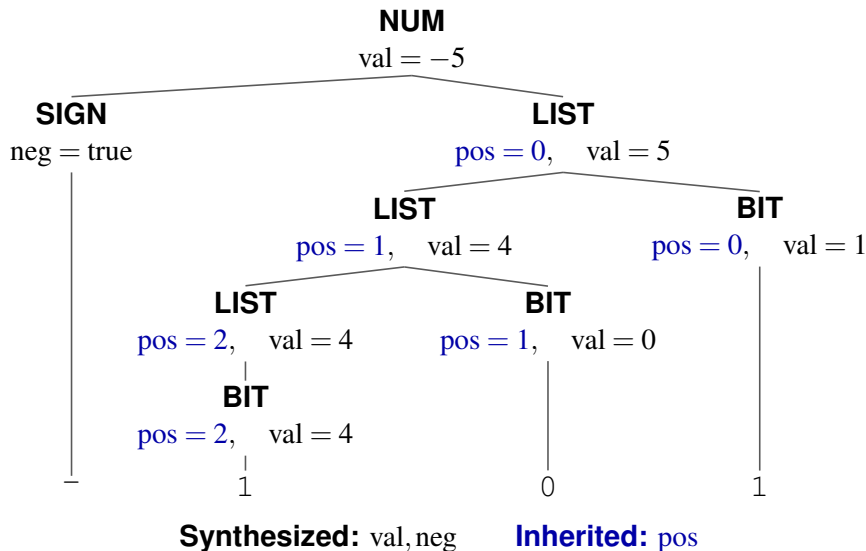
Example: Evaluate signed binary numbers

PRODUCTION	SEMANTIC RULES
NUM $\rightarrow$ SIGN LIST	
SIGN $\rightarrow$ +	
SIGN $\rightarrow$ -	
LIST $\rightarrow$ BIT	
LIST $\rightarrow$ LIST ₁ BIT	
BIT $\rightarrow$ 0	
BIT $\rightarrow$ 1	

Example: Evaluate signed binary numbers

PRODUCTION	SEMANTIC RULES
NUM $\rightarrow$ SIGN LIST	LIST.pos := 0 if SIGN.neg NUM.val := -LIST.val else NUM.val := LIST.val
SIGN $\rightarrow$ +	SIGN.neg := false
SIGN $\rightarrow$ -	SIGN.neg := true
LIST $\rightarrow$ BIT	BIT.pos := LIST.pos LIST.val := BIT.val
LIST $\rightarrow$ LIST ₁ BIT	LIST ₁ .pos := LIST.pos + 1 BIT.pos := LIST.pos LIST.val := LIST ₁ .val + BIT.val
BIT $\rightarrow$ 0	BIT.val := 0
BIT $\rightarrow$ 1	BIT.val := $2^{\text{BIT.pos}}$

Attributed Parse Tree for -101



Dependences between attributes

- values are computed from constants & other attributes
 - *synthesized attribute* – value computed from children
 - *inherited attribute* – value computed from siblings & parent
- *key notion*: induced dependency graph

The attribute dependency graph

- nodes represent attributes
- edges represent flow of values
- graph is specific to parse tree
- size is related to parse tree's size
- can be built alongside parse tree

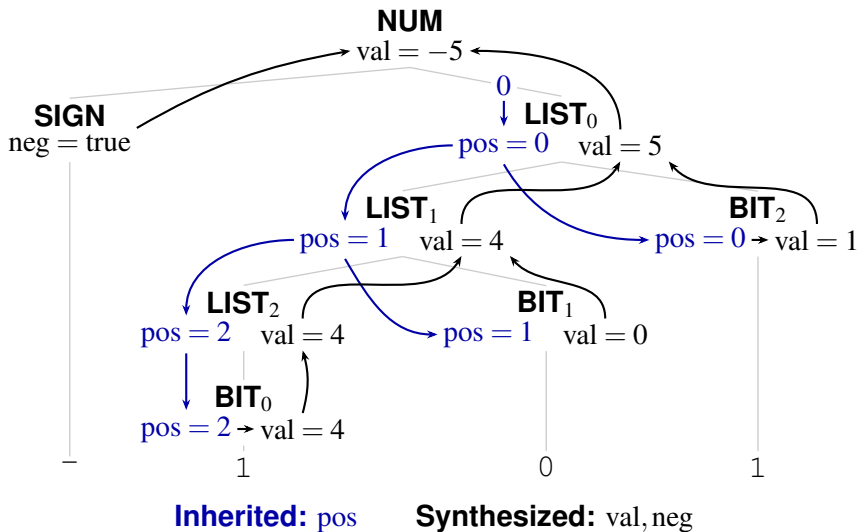
The dependency graph must be acyclic

Evaluation order:

- topological sort the dependency graph to order attributes
- using this order, evaluate the rules

The order depends on both the grammar and the input string

Attribute Dependencies for -101



Example: A topological order

- 1 SIGN.neg
- 2 LIST₀.pos
- 3 LIST₁.pos
- 4 LIST₂.pos
- 5 BIT₀.pos
- 6 BIT₁.pos
- 7 BIT₂.pos
- 8 BIT₀.val
- 9 LIST₂.val
- 10 BIT₁.val
- 11 LIST₁.val
- 12 BIT₂.val
- 13 LIST₀.val
- 14 NUM.val

Evaluating in this order yields NUM.val: -5

Evaluation strategies

- *Parse-tree methods*

(dynamic)

- ① build the parse tree
- ② build the dependency graph
- ③ topological sort the graph
- ④ evaluate it

(cyclic graph fails)

What if there are cycles?

Avoiding cycles

- Hard to tell, for a given grammar, whether there exists any parse tree whose dependency graphs have cycles.
- Focus on classes of SDD's that guarantee an evaluation order – do not permit dependency graphs with cycles.
 - L-attributed – class of SDTs called “L-attributed translations”.
 - S-attributed – class of SDTs called “S-attributed translations”.

L-Attributed Grammars

Informally – allows both synthesized and inherited attributes, but dependency-graph edges may only go from left to right, not other way around.

Given production $A \rightarrow X_1 X_2 \cdots X_n$

- Synthesized attributes of A
- Inherited attributes of X_j depend only on:
 - 1 Inherited attributes of A
 - 2 Arbitrary attributes of $X_1, X_2, \cdots X_{j-1}$

i.e., evaluation order:

- $\text{Inh}(A), \text{Inh}(X_1), \text{Syn}(X_1), \dots, \text{Inh}(X_n), \text{Syn}(X_n), \text{Syn}(A)$
- This is precisely the order of evaluation for an LL parser

L-Attributed Grammar: Examples

PRODUCTION	SEMANTIC RULES
$D \rightarrow T L$	$L.in := T.type$
$T \rightarrow \mathbf{int}$	$T.type := \text{integer}$
$T \rightarrow \mathbf{real}$	$T.type := \text{real}$
$L \rightarrow L_1 , \mathbf{id}$	$L_1.in := L.in$ $\text{addtype}(\mathbf{id}.entry, L.in)$
$L \rightarrow \mathbf{id}$	$\text{addtype}(\mathbf{id}.entry, L.in)$

L-Attributed Grammar: Examples

PRODUCTION	SEMANTIC RULES
NUM $\rightarrow$ SIGN LIST	LIST.pos := 0 if SIGN.neg NUM.val := -LIST.val else NUM.val := LIST.val
SIGN $\rightarrow$ +	SIGN.neg := false
SIGN $\rightarrow$ -	SIGN.neg := true
LIST $\rightarrow$ BIT	BIT.pos := LIST.pos LIST.val := BIT.val
LIST $\rightarrow$ LIST ₁ BIT	LIST ₁ .pos := LIST.pos + 1 BIT.pos := LIST.pos LIST.val := LIST ₁ .val + BIT.val
BIT $\rightarrow$ 0	BIT.val := 0
BIT $\rightarrow$ 1	BIT.val := $2^{\text{BIT.pos}}$

Evaluating attributes of L-attributed grammar

- Perform depth-first traversal starting from the root of the parse tree:

```
depth_first(N) {  
    evaluate inherited attributes of N;  
    for (each child C of N, left to right) {  
        depth_first(C);  
    }  
    evaluate synthesized attributes of N;  
}
```

- Note that this order of visiting nodes corresponds to the exact order in which top-down parser builds the parse tree.
- Thus, we can also evaluate L-attributed grammars in one top-down (LL) pass.

SDT for L-Attributed Grammars

- Recall, embed the action which evaluates an attribute inside the body of the production.

The SDT for $A \rightarrow X_1X_2 \dots X_n$ is

$$A \rightarrow \{\text{INH}(X_1) = \dots\}X_1\{\text{INH}(X_2) = \dots\}X_2 \dots X_n\{\text{SYN}(A) = \dots\}$$

- The action for evaluating an inherited attribute for X is placed immediately before the occurrence of X in the body of the production.
- The action for evaluating a synthesized attribute for A is placed after the entire body of the production.

S-attributed Grammars

- allows only synthesized attributes for non-terminals
- equivalently, semantic actions at far right of a RHS
- Evaluate children before their parent: a **postorder** traversal.
- An LR parser reduces in this same order, so attributes can be evaluated at each reduction.
- One bottom-up (LR) pass suffices. Attribute values live on the parser stack; no explicit parse tree is needed.

Evaluating attributes of S-attributed grammar

- Evaluate it in any bottom-up order of the nodes in the parse tree.
- Apply *postorder* to the root of the parse tree:

```
postorder(N) {  
    for (each child C of N) {  
        postorder(C);  
    }  
    evaluate synthesized attributes of N;  
}
```

- Post order traversal of the parse tree corresponds to the exact order in which the bottom-up parsing builds the parse tree.
- Thus, we can evaluate S-attributed grammars in one bottom-up (LR) pass.

LL parsers and actions

How can we directly evaluate attributes in a L-attributed SDT during LL parsing?

During LL Parsing, we expand productions *before* scanning RHS symbols, so:

- push actions onto parse stack like other grammar symbols
- pop and perform action when it comes to top of parse stack

LL parsers and actions

```
push EOF
push Start Symbol
token  $\leftarrow$  next_token()
repeat
    pop X
    if X is a terminal or EOF then
        if X = token then
            token  $\leftarrow$  next_token()
        else error()
    else if X is an action
        perform X
    else /* X is a non-terminal */
        if  $M[X, \text{token}] = X \rightarrow Y_1 Y_2 \dots Y_k$  then
            push  $Y_k, Y_{k-1}, \dots, Y_1$ 
        else error()
until X = EOF
```

Each Y_i can be:

- a terminal
- a nonterminal
- a **semantic action**

Actions execute
when popped,
without consuming
input.

LR parsers and actions

What about LR parsers?

An LR parser executes an end-of-rule action when it reduces.

- **S-attributed:** actions are already at the end. No transformation is needed.
- **L-attributed:** inherited attributes may require actions before later symbols are parsed.
- For these **mid-rule actions**, introduce a new nonterminal so the action runs at a reduction.

$$A \rightarrow w \text{ action } \beta$$

becomes

$$A \rightarrow M\beta$$

$$M \rightarrow w \text{ action}$$

For more details, refer to Dragon Book, Chapter 5, Section 5.5.4

Inherited and Synthesized Attributes

- **Synthesized attributes carry results upward.**

Examples: expression values, types, and generated code.

- **Inherited attributes carry context into a subtree.**

Examples: declaration environments, expected types, and offsets.

Evaluation order connects attributes to parsing:

- S-attributed definitions fit LR reductions directly.
- L-attributed definitions fit a left-to-right, depth-first traversal.

Pass context in; return results.

Attribute dependencies determine when computations can run.